

Experiment 4: Simulation and Analysis of Wired Network using NS2 Simulator

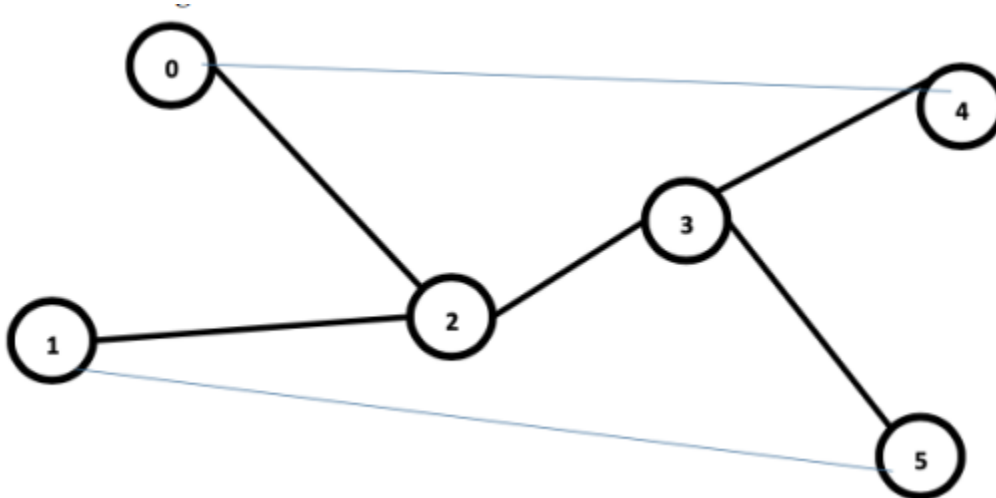
AIM:

To simulate and analyze wired network using NS2 simulator

PROCEDURE:

1. Environment Setup: Install Oracle VM VirtualBox, Linux OS, NS2 simulator, and NAM animation tool on the system.
2. Code Creation: Write the TCL simulation script using any text editor (gedit) and save the file with `.tcl` extension.
3. Network Design: Create 6 nodes (n0 to n5) and establish duplex links with specified bandwidth and delay parameters using DropTail queue management.
4. Agent Configuration: Set up UDP agent with NULL receiver between Node 0 and Node 4, and TCP agent with TCPSink receiver between Node 1 and Node 5.
5. Traffic Generation: Attach CBR (Constant Bit Rate) application to UDP agent and FTP (File Transfer Protocol) application to TCP agent.
6. Simulation Schedule: Schedule traffic start times (CBR at 1.0s, FTP at 2.0s) and define finish procedure to end simulation at 10.0 seconds.
7. Execution and Analysis: Run the simulation using `ns filename.tcl`, view animation using `nam filename.nam`, and analyze trace file using `gedit filename.tr` for performance metrics.

NETWORK DESIGN:



Design Procedure:

Node 0 – Act as an agent and can send the UDP packet at CBR mode.

Node 1 – Act as an agent and can send TCP packet at FTP mode.

Node 4 – Receives the packet from Node 0. It is going to receive UDP packet hence it is NULL agent.

Node 5 – Receives the packet from Node 1. It is going to receive TCP packet hence it acts as Sink Agent.

Link capacity between Node 0 to Node 2 is 5Mbps bandwidth and 2ms speed.

Link capacity between Node 1 to Node 2 is 10Mbps bandwidth and 5ms speed.

Link capacity between Node 2 to Node 3 is 4Mbps bandwidth and 3ms speed.

Link capacity between Node 3 to Node 4 is 100Mbps bandwidth and 2ms speed.

Link capacity between Node 3 to Node 5 is 15Mbps bandwidth and 4ms speed.

CODE:**sim.tcl**

```
#Create a simulator objectset  
set ns [new Simulator]
```

```
#Create a trace file, this file is for logging purpose  
set tracefile [open wired.tr w]  
$ns trace-all $tracefile
```

```
#Create a animation information or NAM file creationset  
set namfile [open wired.nam w]  
$ns namtrace-all $namfile
```

```
#Create nodes  
set n0 [$ns node]  
set n1 [$ns node]  
set n2 [$ns node]  
set n3 [$ns node]  
set n4 [$ns node]  
set n5 [$ns node]
```

#Creation of link between nodes with DropTail Queue

#Droptail means Dropping the tail.

\$ns duplex-link \$n0 \$n2 5Mb 2ms DropTail

\$ns duplex-link \$n1 \$n2 10Mb 5ms DropTail

\$ns duplex-link \$n2 \$n3 4Mb 3ms DropTail

\$ns duplex-link \$n3 \$n4 100Mb 2ms DropTail

\$ns duplex-link \$n3 \$n5 15Mb 4ms DropTail

#Creation of Agents

#Node 0 to Node 4

set udp [new Agent/UDP]

set null [new Agent/Null]

\$ns attach-agent \$n0 \$udp

\$ns attach-agent \$n4 \$null

\$ns connect \$udp \$null

#Creation of TCP Agent

set tcp [new Agent/TCP]

set sink [new Agent/TCPSink]

\$ns attach-agent \$n1 \$tcp

\$ns attach-agent \$n5 \$sink

\$ns connect \$tcp \$sink

#Creation of Application CBR, FTP

#CBR - Constant Bit Rate (Example mp3 files that have a CBR or 192kbps, 320kbps, etc.)

#FTP - File Transfer Protocol (Ex: To download a file from a network)

set cbr [new Application/Traffic/CBR]

\$cbr attach-agent \$udp

set ftp [new Application/FTP]

\$ftp attach-agent \$tcp

#Start the traffic

\$ns at 1.0 "\$cbr start"

\$ns at 2.0 "\$ftp start"

\$ns at 10.0 "finish"

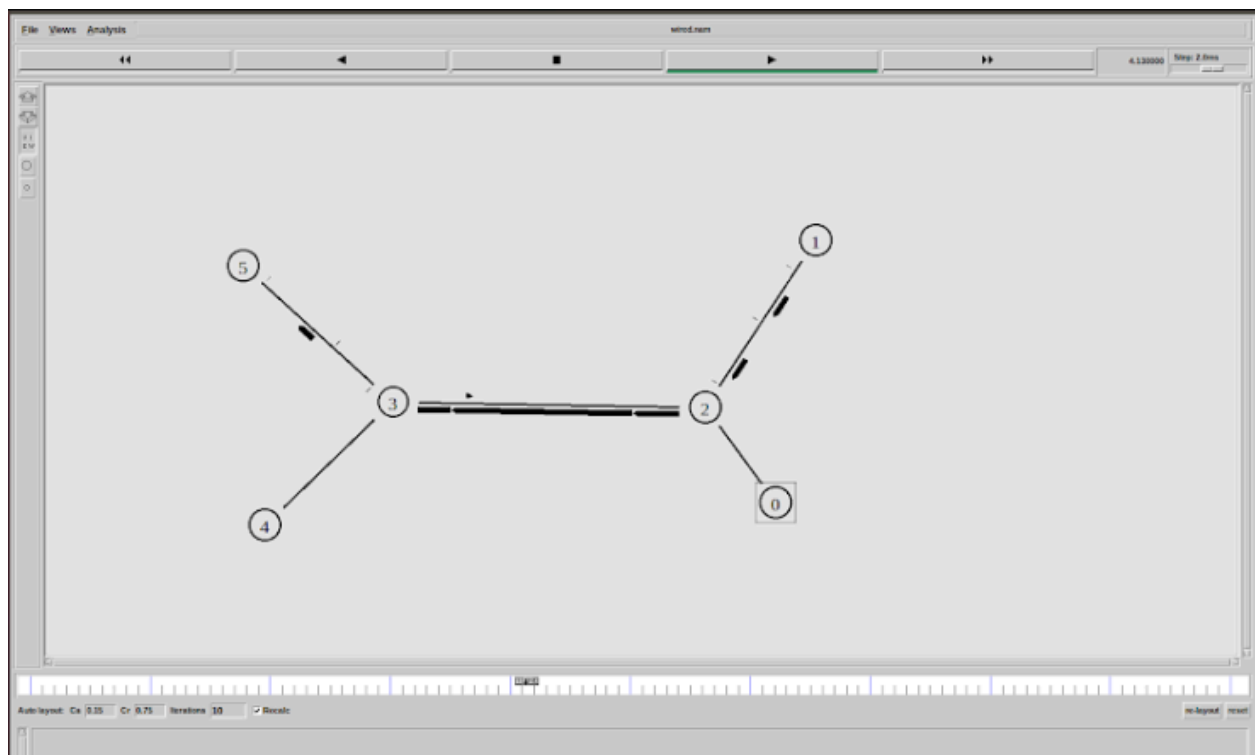
#The following procedure will be called at 10.0 seconds

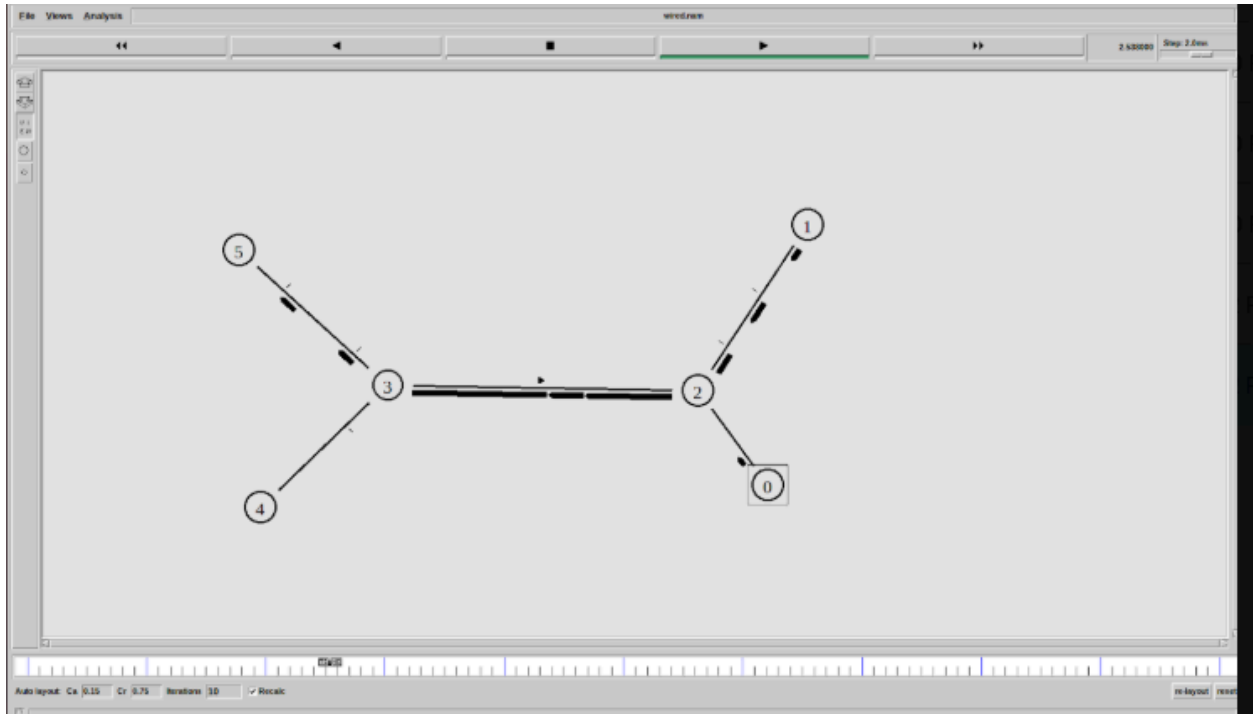
proc finish {} {

```
global ns tracefile namfile
$ns flush-trace
close $tracefile
close $namfile
exit 0
}
```

```
puts "Simulation is starting..."
$ns run
```

OUTPUT:





RESULTS:

UDP traffic maintained constant throughput of 4 Mbps with zero packet loss throughout the simulation, while TCP throughput dropped from 10 Mbps to approximately 1.5-2 Mbps after 2 seconds due to congestion, with packet drops reaching 35-40% at the Node 2-Node 3 bottleneck link (4 Mbps bandwidth).

INFERENCE:

UDP's lack of congestion control allows it to dominate bottleneck bandwidth, causing unfair resource allocation and severe performance degradation for TCP traffic which employs congestion avoidance mechanisms, demonstrating that TCP sacrifices throughput for reliability while UDP prioritizes constant transmission rate over network fairness.